(Luca Trevisan) Checking the Quasirandomness of Graphs and Hypergraphs

Terence Tao · 15 February 2008

https://terrytao.wordpress.com/2008/02/15/luca-trevisan-checking-the-quasirandomness-of-graphs-and-hypergraphs/

tao2typ · converted to Typst 2026-10-14 (UTC)

[*This post is authored by Luca Trevisan. – T.*]

Notions of "quasirandomness" for graphs and hypergraphs have many applications in combinatorics and computer science. Several past posts by Terry have addressed the role of quasirandom structures in additive combinatorics. A recurring theme is that if an object (a graph, a hypergraph, a subset of integers, ...) is quasirandom, then several useful properties can be immediately deduced, but, also, if an object is not quasirandom then it possesses some "structure" than can be usefully exploited in an inductive argument. The contrapositive statements of such randomness-structure dichotomies are that certain simple properties imply quasirandomness. One can view such results as algorithms that can be used to "certify" quasirandomness, and the existence (or, in some cases, conjectural non-existence) of such algorithms has implications in computer science, as I shall discuss in this post.

Informally, an undirected graph $G = (V, E)$ is *quasirandom* if, for every two large sets of vertices S and T, the number of edges between S and T is approximately what it would be, on average, if we had generated G by picking $|E|$ edges uniformly at random. Quantitatively, we'll say that a graph $G = (V, E)$ is ϵ-*quasirandom* if for every two disjoint sets of vertices S and T the number of edges between S and T differs from $|E| \cdot |S| \cdot |T| / \binom{|V|}{2}$ by at most an $\epsilon |E|$ additive error term. This property is true with $\epsilon = 0$ for the complete graph, and it is quite remarkable that there exist very sparse graphs that are quasi-random. Indeed it is possible to show that a random graph with $O(|V|/\epsilon^2)$ edges (that is, a graph where the average degree of a vertex is a constant depending only on ϵ) is ϵ-quasirandom with high probability. While this is remarkable enough, it is also true that there are efficient algorithms that, given a random graph with $c|V|$ edges, are able with high probability to verify that the graph is $O(1/\sqrt{c})$-quasirandom. (This is true with some caveats, as explained below.)

Here are some implications of such algorithms. Consider the problem of finding a largest independent set in a given graph. (If vertices represent tasks to be performed, and an edge represents a conflict that makes it impossible to perform two tasks at the same time, a largest independent set is a largest choice of simultaneously doable tasks.) This problem is NP-hard, meaning that if there were an efficient optimization algorithm for it, there would be an algorithm that given a provable mathematical statement would find a shortest proof for it in time polynomial in the length of the shortest proof; a number of other highly unlikely algorithmic consequences, including polynomial time algorithms to break any possible cryptographic protocol, would also follow. (See e.g. this description of the P versus NP problem.) In addition, we know that, in an n-vertex graph, even achieving an $n^{.99}$ approximation in polynomial time would imply that $P = NP$ (as proved by Hastad) and that, in particular, exact optimization would also be possible in polynomial time. When we look at graphs of maximum degree d, a $d+1$ approximation is trivial, but it is NP-hard to approximate the problem better than $d/2^{\sqrt{\log d}}$ (see this paper).

Now consider a random d-regular graph; it is easy to find an independent set of size $|V|/(d+1)$, and, with high probability, we can prove that the graph is $O(1/\sqrt{d})$-quasirandom, and hence the largest independent set has size $O(|V|/\sqrt{d})$. This means that we

achieve with high probability an approximation of $\sqrt{d}$, and so that

random graphs are not the hardest instances of the problem. Typically, in combinatorics, when random graphs are not extremal for a given problem, then finding extremal constructions is very hard; indeed,

one has to work quite hard to come up with examples showing that certain approximate algorithms for the independent set problem (such as the Lovasz theta function) do achieve approximation as bad as $d^{1-o(1)}$ (see this paper by Feige).

The same phenomenon occurs for a number of other graph optimization problems. For example the Max Cut problem is the problem of finding a partition of the vertices of a given graph into two sets, so that a largest number of edges are "cut," that is, have endpoints on different sides of the partition. There is a polynomial-time algorithm that achieves an $\approx .878$ approximation factor, it is NP-hard to approximate the problem better than $16/17$, and there is some evidence that no approximation better than $\approx .878$ is achievable (see Section 5.1 of this paper for references). On the other hand, in every graph it is easy to find a partition that cuts half of the edges (use a random partition), and in an ϵ-quasirandom graph no partition can cut more than a $1/2 + O(\epsilon)$ fraction of edges; this means that, here, too, when looking at worst-case instances for certain algorithms one must resort to non-random constructions.

Having motivated the problem, let me briefly describe the algorithms.

First I'd like to discuss an algorithm that gives slightly worse bounds and works only on dense graphs, but is surprisingly simple: count the number of 4-cycles in the graph. This amazing result is due to Chung, Graham, Wilson and Thomason (see the introductory sections of this paper of Gowers for further references). Suppose we have a graph with n vertices and $p\binom{n}{2}$ edges; then a first observation is that the number of 4-cycles (that is, the number of 4-tuples of vertices (x_1, y_1, x_2, y_2) such that the four edges (x_i, y_j) exist for $i, j \in \{1, 2\}$) is at least $p^4 n^4$, which follows from two applications of Cauchy-Schwarz:

$$\begin{aligned} n^8 p^4 &= (\textstyle\sum_{x,y} A(x,y))^4 \\ &\leq (n \textstyle\sum_{x,y_1,y_2} A(x,y_1)A(x,y_2))^2 \\ &\leq n^4 \textstyle\sum_{x_1,x_2,y_1,y_2} A(x_1,y_1)A(x_1,y_2)A(x_2,y_1)A(x_2,y_2) \end{aligned}$$

where we define $A(x, y) := 1$ if (x, y) is an edge, and $A(x, y) := 0$ otherwise.

Note that $p^4 n^4$ is the average number of 4-cycles in a random graph with n vertices in which each edge is included with probability p. The surprising thing is that if, in a given graph, the number of 4-cycles is close to $p^4 n^4$, then the graph is quasirandom. This follows from applying Cauchy-Schwarz twice, in a similar way, starting from the expression

$$\frac{1}{n^4}\Big(\sum_{x\in S, y\in T} (A(x,y) - p)\Big)^4$$

which we want to prove to be at most $\epsilon^4 p^4 n^4$ for every set S and T, in order to establish that the graph is ϵ-pseudorandom. It turns out that the above expression is upper bounded by the difference between the number of 4-cycles in the graph and $p^4 n^4$.

When we deal with sparse graphs, unfortunately, this approach breaks down. In fact, any approach that looks at small patterns is bound to fail: a random graph with $c|V|$ edges and a random bipartite graph with $c|V|$ edges look "locally the same" even though the former is very likely to be $O(1/\sqrt{c})$-quasirandom, while the latter is not quasirandom at all.

For sparse *regular* graphs (for which all vertices have the same degree), "spectral" techniques give extremely good certificates of quasirandomness. Let $G = (V, E)$ be a regular graph where all vertices have degree d, and A be the adjacency matrix of G, hence $A(u, v) = 1$ if (u, v) is an edge and $A(u, v) = 0$ otherwise. One can see that:

1. A is symmetric, hence all eigenvalues are real;
2. the vector $(1, ..., 1)$ is an eigenvector with eigenvalue d;
3. all eigenvalues are between -d and d. Let now $\lambda_1 \geq \lambda_2 \geq ... \geq \lambda_n$ be the eigenvalues of A, with multiplicities; as we said, $\lambda_1 = d$.

It turns out that if all the other eigenvalues are at most ϵd in absolute value, then G is ϵ-quasirandom. More precisely, if S and T are disjoint sets, then

$$\left| e(S,T) - \frac{d \cdot |S| \cdot |T|}{|V|} \right| \leq \max\{|\lambda_2|, |\lambda_n|\} \cdot \sqrt{|T| \cdot |V|}$$

This result is known as the *expander mixing lemma*, and the intuition of the proof is quite simple. Let $\mathbf{v}_1, ..., \mathbf{v}_n$ be a system of orthonormal eigenvector for the eigenvalues $\lambda_1, ..., \lambda_n$. The number of edges between the sets S and T is

$$1_S A 1_T^t$$

where, for a set S, we define 1_S as the vector such that $1_S(u) = 1$ if $u \in S$ and $1_S(u) = 0$ otherwise. This can be rewritten as

$$d \cdot 1_S \cdot \mathbf{v}_1^t \cdot 1_T \cdot \mathbf{v}_1^t + \sum_{i=2}^{n} \lambda_i 1_S \mathbf{v}_i^t \cdot 1_T \cdot \mathbf{v}_i^t$$

and the first term is $d|S||T|/n$, while the other terms can be upper bounded by $\max_{i=2,...,n} |\lambda_i| \sqrt{|S||T|}$ in absolute value.Furthermore, if one picks a d-regular graph at random, then all eigenvalues except λ_1 are at most $O(\sqrt{d-1})$ with high probability. One can in fact prove that the upper bound is close to $2\sqrt{d-1}$, which is the best possible, but such arguments are very difficult. (A nitpick here is that said results apply to the probabilistic model where we take the union of d random matchings rather than a random d-regular graph.)

To sum up, if a random graph is generated by taking the union of d random matchings, we can certify that is is $O(1/\sqrt{d})$-quasirandom. In the $G_{n,p}$ model where each edge is selected with probability p, similar arguments apply (and can certify $1/O(\sqrt{d})$-quasirandomness, where $d = pn$ is the average degree), provided that the average degree pn is at least $(\log n)^{O(1)}$. The case of average constant degree is trickier, but it can probably be handled by standard techniques (removing high-degree and isolated vertices before computing the spectrum) that have succeeded in related problems.

Given how satisfactory are the combinatorial and algorithmic theories of quasirandomness for graphs, we may hope for similar results for hypergraphs.

In the dense case, an approach based on counting local patterns indeed works in a completely analogous way. In a 3-uniform hypergraph (an hypergraph where each hyperedge contains three vertices) with n vertices and $p\binom{n}{3}$ hyperedges we shall count the number of 6-tuples $(x_1, x_2, y_1, y_2, z_1, z_2)$ of vertices such that all the 8 hyperedges of the form (x_i, y_j, z_k) exist for $i, j, k \in \{1, 2\}$. Three applications of Cauchy-Schwarz show that this number is at least $p^8 n^6$, and that if this number is at most $(1 + \epsilon^{O(1)})p^8 n^6$ then the hypergraph is ϵ-quasirandom.

Again, however, local approaches fail for sparse hypergraphs.

An algorithm for the sparse case would have remarkable applications for the kSAT problem. In the kSAT problem we are given a boolean AND-of-ORs expression where each OR term is over k variables, for example this is an instance of 3SAT:

$$(x_1 \vee x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee \neg x_4)$$

and we want to know if there is a setting to the variables that satisfies the formula. In the above example, setting all variables to True works. This problem is a canonical NP-complete problem for all $k \geq 3$: if a polynomial time algorithm existed, then there would be polynomial time algorithms for all problems in NP. (There is a linear time algorithm to solve 2SAT.)

Recently, there has been considerable interest in the question of the average-case complexity of kSAT, in the model where we pick at random a formula with n variables and $m(n)$ clauses. (A "clause" is an OR term in the formula.)

In the hard graph problems we discussed above, uniformly chosen random instances were "easier" than worst-case instances, assuming $P \neq NP$. In the kSAT case, the situation seems more complex. It is conjectured that, for every k, there is a constant c_k such that when we generate a random formula with $m(n) \leq (c_k - \epsilon)n$ clauses then the formula is satisfiable with high probability, and when we generate

a random formula with $m(n) \geq (c_k + \epsilon)n$ then the formula is unsatisfiable with high probability. Furthermore, there are algorithms that are conjectured to find satisfying assignments with high probability in the $m(n) \leq (c_k - \epsilon)$ regime. (See my posts here, here and here for references).

An open question is whether there is an efficient way to certify the unsatisfiability of random formulas in the $m(n) \geq (c_k + \epsilon)n$ regime.

One approach in this direction is to reduce to an hypergraph problem. Given a kSAT formula with n variables and m edges, construct a k-uniform hypergraph with 2n vertices (one for each variable x and bit b, which we think of as being the assignment $x = b$) and m hyperdges, one for each clause, connecting the unique values of the variables that contradict the clause. Suppose the formula has a satisfying assignment, then there is a set of half the vertices such that no hyperedge is completely contained in the set; this is called an independent set. Now, if one generates the formula randomly, then the associated hypergraph is a random hypergraph. The question then becomes: given a random hypergraph, can we certify that there is no large independent set? If we could certify that a random hypergraph of sufficiently large average degree is, say, $1/100$-quasirandom, then the non-existence of large independent sets would follow, and we would be able to certify the unsatisfiability of comparably dense random kSAT instances.

A natural approach to certify the quasirandomness of a given sparse hypergraph is to look for generalizations of spectral methods. While there is no clear definition of what the "eigenvalues" of an 3-hypergraph (or, more generally, of a symmetric trilinear form) should be, Friedman and Wigderson consider the characterization of the second eigenvalue of a graph as the problem of optimizing a certain quadratic form. The problem has a natural generalization to trilinear forms, and yields a definition of "second eigenvalue" of a 3-hypergraph. They are able to prove that this parameter is bounded for random hypergraphs and that when it is bounded the hypergraph is quasirandom. Unfortunately, no algorithm is known to efficiently compute this parameter.

It is becoming a standard conjecture that, indeed, the unsatisfiability of random $kSAT$ instances is hard to verify when $m(n) = cn$ and c is an arbitrarily large constant. (Note that the problem becomes easier, or no harder, when c is larger.) On the other hand, simple refutation algorithms exist for kSAT in the setting of very high density, and the Chung-Graham-Wilson approach can certify quasirandomness of dense hypergraphs.

We can then ask what is the minimal density for which such problems admit efficient algorithmic solutions.

For kSAT, so far, the best refutation algorithms work when the density is about $n^{k/2}$, and, for even k, the idea of such algorithms also leads to an algorithm to verify the quasirandomness of random k-hypergraphs.

Translated to the hypergraph setting, the algorithm is as follows. Given a k-hypergraph (even k) with n vertices and about $n^{k/2}$ hyperdges, we create a graph with $n^{k/2}$ vertices, one per $k/2$-tuple. Each k-hyperedge is associated to an edge in the graph (by splitting, say, randomly, the k-tuple into two $k/2$ -tuples). One can see that if the graph is ϵ-quasirandom, then the hypergraph is $O(\epsilon)$-quasirandom. (The contrapositive is easier to prove, suppose the hypergraph is not quasirandom, let $S_1, ..., S_k$ be the sets that witness that the quasirandomness property is contradicted; then the sets $(S_1 \times \cdots \times S_{k/2})$ and $(S_{k/2+1} \times \cdots \times S_k)$ contradict the quasirandomness of the graph.) If the hypergraph is randomly generated, then the graph is randomly generated, and we can use spectral techniques to verify its quasirandomness.

For odd k, there are algorithms that certify the unsatisfiability of random kSAT formulas with about $n^{k/2}$ clauses; as far as I know, the analogous result is open for hypergraphs. (The kSAT algorithms use properties of kSAT that have no analog in the general hypergraph setting, such as the fact that an assignment that satisfies both $(x \vee y \vee q)$ and $(\neg q \vee w \vee z)$ must also satisfy $(x \vee y \vee w \vee z)$.

Substantial work has gone into proving limitations of spectral methods for the problem of certifying the unsatisfiability of random instances of kSAT. Bounds at least as good as those obtained by spectral

methods can always be obtained by formulating a semidefinite programming (SDP) relaxation of the given problem, and there are several results showing that even very powerful SDP relaxations of kSAT cannot certify unsatisfiability even for fairly large densities (see e.g. this paper), and the hypergraph quasirandomness problem is only harder. Still, it is plausible that one can verify the quasirandomness of a random hypergraph with at least $n^{1+\epsilon}$ hyperedges, for all $\epsilon > 0$. At least, the problem should be solvable in 3-hypergraphs with $n^{1.5}$ hyperedges.

Comments (20)



Terence Tao · 15 February 2008

I have a problem that I can't solve relating to the certifiability of hypergraph quasirandomness which is related to Luca's post, and which potentially has various applications to breaking the parity problem in number theory.

It goes as follows. Let $[n] := \{1, ..., n\}$ for some large integer n. We consider two functions $f, g : [n]^2 \to [n]^2$ from the square to the square (which can be identified with two 4-uniform 4-partite hypergraphs on 4 groups of n vertices). The first function f is a random permutation from $[n]^2$ to $[n]^2$. The second function g is slightly harder to describe. One first needs a random colouring $c : [n] \to \mathbb{Z}/2\mathbb{Z}$, and then g is a random permutation subject to the additional property that $c(x) + c(y) = c(z) + c(w) \mod 2$ whenever $g(x, y) = (z, w)$. To put it another way, the colouring c divides $[n]^2$ in a "checkerboard fashion" into "white squares" $\{(x, y) \in [n]^2 : c(x) + c(y) = 0\}$ and two "black squares" $\{(x, y) \in [n]^2 : c(x) + c(y) = 1\}$, and g is a random permutation from white squares to white squares and from black squares to black squares.

My problem is as follows: suppose one is given the two functions f and g (but not c) as above, but is not told which is which. Can one build an efficient distinguisher that can determine which function is f and which is g with high probability?

I know that any purely "local" statistic (e.g. number of "octahedra") cannot distinguish f from g; some sort of "eigenvalue" computation seems to be needed instead. Unfortunately, f and g seem to be too sparse (only n^2 hyperedges among n^4 possible hyperedges) to get anywhere. Note that the hypergraph associated to f is quasirandom, whilst the hypergraph associated to g is not, so this problem can be viewed as a special case of Luca's general question.



Klas M. · 16 February 2008

Dear Terry,

Would not the number of cycles distinguish g from f with high probability?

A random permutation of an n-set has an expectation of $\log n$ cycles, and small variance. The permutation g is built from independent permutations of two sets, both of size around $n^2/2$, and f is a random permutation of a set of size n^2, so g is expected to have about twice as many cycles as f.



Terence Tao · 16 February 2008

Hmm, fair enough, that solves my problem as stated :-) . I'll have to think about whether that solution extends to the more complicated situations for which this problem was a toy model.

Suppose though that we change things a little bit. Instead of one colouring map $c : [n] \to \mathbb{Z}/2\mathbb{Z}$, we have four maps $c_1, c_2, c_3, c_4 : [n] \to \mathbb{Z}/2\mathbb{Z}$, chosen independently at random (and all unknown to the distinguisher). Let's say now that g is random subject to the constraint that $c_1(x) + c_2(y) = c_3(z) + c_4(w)$ whenever $g(x, y) = (z, w)$. It seems now that f and g have the same number and distribution of cycles, so the distinguisher problem needs a different attack in this case.



Chris · 17 February 2008

Dear Prof. Tao, I'm not sure if I understand your problem
correctly, but consider the following.
I assume the distribution of random colourings c_1, c_2, c_3, c_4 is uniform on the set $C_{1234} = \{c_1, c_2, c_3, c_4 : \{\#\{(x,y) : c_1(x) + c_2(y) = i\} = \#\{(z,w) : c_3(z) + c_4(w) = i\}; i = 0, 1\}\}$.
I need some notation for partitionings of $[n]^2$, namely let $P_i^{k,m} = \{(x,y) : c_k(x) + c_m(y) = i\}$.
Consider the following (very similar) procedures of sampling such colourings.
1. Choose uniformly at random c_1 and independent of c_1 choose uniformly at random c_2. Now, independent of c_1 and c_2 choose a random permutation, say h, from $[n]^2$ to $[n]^2$ and let $C_i^{3,4} = h(P_i^{1,2})$, for $i = 0, 1$. Now, if $(z,w) \in C_1^{3,4}$ toss a coin to decide between $c_3(z) = 1$; $c_4(w) = 0$ and $c_3(z) = 0$; $c_4(w) = 1$. If $(z,w) \in C_0^{3,4}$ toss a coin to decide between $c_3(z) = c_4(w) = 1$ and $c_3(z) = c_4(w) = 0$. Clearly after we defined c_3 and c_4 also $P_i^{3,4} = C_i^{3,4}$.
2. Choose uniformly at random c_1 and independent of c_1 choose uniformly at random c_2. Let $K_0 = \#P_0^{1,2}$ Now, independent of c_1 and c_2 choose a random partitioning of $[n]^2$, into $C_0^{3,4}$ and $C_1^{3,4}$ such that $K_0 = \#C_0^{1,2}$, the partitioning depends on K_0, but not on other features of c_1, c_2. Now, if $(z,w) \in C_1^{3,4}$ toss a coin to decide between $c_3(z) = 1$ $c_4(w) = 0$ and $c_3(z) = 0$ $c_4(w) = 1$. If $(z,w) \in C_0^{3,4}$ toss a coin to decide between $c_3(z) = c_4(w) = 1$ and $c_3(z) = c_4(w) = 0$. Clearly after we defined c_3 and c_4 also $P_i^{3,4} = C_i^{3,4}$. The last step is to choose uniformly at random mappings, say h_0, from $P_0^{1,2}$ to $P_0^{3,4}$ and h_1, from $P_1^{1,2}$ to $P_1^{3,4}$ and glue them to obtain h.
The observation is that in both sampling schemes h follows the same distribution (and in fact $\{c_1, c_2, c_3, c_4, h\}$ follow the same distribution) whereas the h defined in (1) is f and
the h defined in (2) is g.
Thus, if the sampling schemes are correct, f and g are statistically indistinguishable.



Terence Tao · 17 February 2008

Dear Chris,
The problem is that $c_3(z)$ and $c_4(w)$ are functions of only one of the two variables (z,w), and so one cannot always consistently specify these functions as (z,w) ranges across, say, $C_1^{3,4}$. For instance suppose (z,w) and (z,w') both lie in $C_1^{3,4}$; the coin tosses for each pair may lead to inconsistent prescriptions of what $c_3(z)$ should be.
The easiest way to show that f and g are distinguishable is to see that the graph of f (as viewed as a sparse 4-uniform 4-partite hypergraph on 4n vertices) is quasirandom in the sense of the main post (in particular, for any sets A, B, C, D of size n/2 in [n], the number of quadruples (x,y,z,w) in A x B x C x D such that f(x,y)=(z,w) is always close to the expected number, which is n^2/16), the graph of g is not (just take A,B,C,D to be the sets where c_1=0, c_2=0, c_3=0, c_4=0 respectively).



Chris · 17 February 2008

I see, my bad! :)



Jun Tarui · 17 February 2008

Hi.
For both the original and revised problems, can't we distiguish f and g with high probability in the following way?
Fix an arbitray O(log n) by O(log n) square. Focusing on this square, check if there are balanced colorings with respect to which a function in question preserves the parity, where coloring is balanced if the number of each color doesn't exceed, say, 2/3 fraction. Do naive, exhaustive checking; the number of all colorings (balanced or not) is poly(n).

Random coloring is balanced with probability at least 1-1/poly(n). The probability that a random permutation has any parity-preserving colorings is at most n^-Omega(log n) by a union bound over all balanced colorings.

 **Terence Tao** · 17 February 2008

Dear Jun Tarui,
I initially thought this worked (and it would be a great solution!) but unfortunately the problem is that f and g don't map small squares to small squares; a O(log n) x O(log n) square gets mapped all over the place inside $[n]^2$, and in fact generically one doesn't even expect the image to ever repeat any coordinates (one is far below the threshold for the birthday paradox to come into play). So one can't detect parity preservation inside such a square.

 **hikolapola** · 18 February 2008

thanks for such a usefull post!

 **Jun Tarui** · 18 February 2008

Dear Terry,
Thank you for explaining my misunderstanding in my previous post above. Let me try again. For simplicity, I'll consider random balanced colorings, i.e. ones that give each color to n/2 points, and general vs parity-preserving random *functions* from [n]^2 to [n]^2 as opposed to permutations on [n]^2.
Check if there are four points that form a grid-parallel rectangle and get mapped to a (possibly degenerate) rectangle with diagonal pairs going to diagonal pairs. If such four points exist, declare that a function is parity-preserving g, and f otherwise. I think this distinguishes random f vs parity-preserving random g with constant probability. (You say above *I know that any purely "local" statistic (e.g. number of "octahedra") cannot distinguish f from g*, so maybe I'm again missing something; sorry if this is the case...)
Consider an arbitrary rectangle R. Prob[f(R) is a rectangle] is about 2/n^4. For g, R is either (1) monochromatic or (2) bichromatic with vertical/horizontal/diagonal pairs having the same color; in all cases, Prob[g(R) is a rectangle] is about 4/n^4, i.e., twice as large. The number of rectangles is (n choose 2)^2, i.e., about n^4/4. So the expected number of rectangles mapping to a rectangle is about 1/2 for f and about 1 for g.
We want to apply a Poisson approximation argument (Alon-Spencer: The probabilistic method, Chapter 8). So consider the probability of two rectangles R1 and R2 both getting mapped to rectangles. Consider the size of their intersection, i.e. how many points belong to both R1 and R2; it is either 0 or 1 or 2. If it is 0 or 1, we can consider the events of R1/R2 getting mapped to a rectangle as independent for both cases of f and g. If it is 2, the intersection forms a vertical or horizontal pair, and the probabilities for f and g respectively are about 2/n^7 and 8/n^7; so they are O(1/n^7). But the number of such pairs (R1, R2) is O(n^5), and thus their contribution for the sake of Poisson approximation ("Delta" in Alon-Spencer) is only O(1/n^2).
So for any balanced coloring, under a parity-preserving random permutation g, the probability that at least one rectangle gets mapped to a rectangle is 1 – e^(-1+O(1/n^2)), i.e., about 1-1/e, whereas for a random permutation f, the probability is at most about 1/2 since the expectation is about 1/2.

 **Terence Tao** · 18 February 2008

Dear Jun Tarui,
Your analysis looks correct, and does provide a probabilistic distinguisher between f and g that has a greater than 50% chance of being accurate, though it does not seem obvious how to iterate it and create a distinguisher which is accurate with arbitrarily high probability for any fixed f and g. (At the

risk of seeming to constantly move the goal posts, I actually want a little bit more than this: I want a rigorous certificate of quasirandomness which cannot be satisfied by any g, but is satisfied by most f. But a 99% accurate distinguisher would already be progress, and more than I am currently able to do.) I guess I should have qualified my previous assertion on equivalence of local statistics; my analysis was only for those patterns which occurred quite frequently (at least n^c of them for some c > 0). In such cases, the number of independent variables exceeds twice the number of equations and so the parity constraints have only a negligible effect. But as you point out, the statistics do shift a little bit for those patterns which only occur O(1) times.

 **Chris** · 18 February 2008

Dear Prof. Tao,
is it OK to assume I can compute $g^{-1}(z,w)$ and $f^{-1}(z,w)$ in constant time?
Also, what do you mean by efficient distinguisher in your initial post? Since n is large, do you expect it to take poly(log(n)) to compute it? or linear of n or maybe poly(n) is OK?

 **Terence Tao** · 19 February 2008

Dear Chris,
The best distinguisher I know that accepts all g and rejects most f has exponential cost in n, so anything polynomial in n or even subexponential would be an improvement. (In particular this means that lookup tables for f^{-1} and g^{-1} are essentially free.)

 **Jun Tarui** · 19 February 2008

Dear Terry,
Just one note to your remark (in your comment 1) "Unfortunately, f and g seem to be too sparse (only n^2 hyperedges among n^4 possible hyperedges) to get anywhere. "
If there are cn^2 hyperdedges for sufficiently large constant c, I think it suffices to certify non-quasirandomness of g by the procedure explained in the last part of the main post. In particular, for your revised problem, given 16 independent copies of f and g (independent random g that are parity-preserving with respect to fixed colorings), I think we can distinguish them with high probability.

 **Jun Tarui** · 20 February 2008

Dear Terry,
I think that the following solves your revised problem as stated (without certifying non-quasirandomness).
Let h be a permutation in question. For x, y, z, w in [n], if h(x, y) = (z, w), write the equation c1(x) + c2(y) = c3(z) + c4(w); thus obtain n^2 mod-2 linear equations in 4n unknowns. Solve the linear system to see if it has a solution such that at least one of c1, c2, c3, c4 is not constant, i.e., not all-0 nor all-1. (This is equivalent to the kernel having dimension at least 4; it can be easily checked.)
With high probability, a random permutation f does not have such a solution: There are 2^(4n) possible colorings(=solutions); for each coloring, the probability that a random permutation respects the coloring is smaller than n^(-Omega(n)).

 **Terence Tao** · 20 February 2008

Dear Jun Tarui,
That is a nice solution to the problem! The one drawback (for the applications I have in mind) is that it is not too robust with respect to noise; if for instance the parity condition is violated by g for o(n^2) of the pairs x,y then it is not clear how to do "robust linear algebra over F_2" and make the distinguisher work in this case. (In other words, as you say, it is not a quasirandomness certificate.)

Though if the noise was much smaller, say only O(log n) errors, then presumably your earlier random sampling trick would work to eradicate it.
(I'm also interested in the case when the functions f, g are only partially defined, say on 1% of the pairs x,y, and possibly multi-valued and non-injective (e.g. each (z,w) could be hit by as many as 100 (x,y)). But any reasonably robust distinguisher for the permutation case should extend to this more general case – for instance, your linear algebra distinguisher seems to work in this case as long as there is only a negligible amount of noise.)

 **Anon** · 21 February 2008

A minor correction: the number of copies of the 4-cycle in a graph with density p it at least p^4n^4 and not at most p^4n^4.

 **Terence Tao** · 21 February 2008

Thanks for the correction!
Best,
Terry

Local Events, Turan's Problem and Limits of Graphs and Hypergraphs « Combinatorics and more · 25 May 2008

[…] Related posts: "Extremal Combinatorics I," and over Terry Tao's blog: (Luca Trevisan) Checking the Quasirandomness of Graphs and Hypergraphs […]

The extremal utility belt: Cauchy-Schwarz « Portrait of the Mathematician · 10 March 2010

[…] extremal utility belt: Cauchy-Schwarz By Harrison I found this little gem in an old post on Terry Tao's blog. There's not really enough content in it to merit an entire […]